

DSAI/S2_BigData/DSAI_HDB_Project_ML_Backup/resale_flat_dagster/resale_flat_dagster/assets/ml_preprocessor.py

```

1 # resale_flat_dagster/resale_flat_dagster/assets/ml_preprocessor.py
2 import pandas as pd
3 import numpy as np
4
5 def build_production_features(df, rpi_dict=None):
6     """Processes final production warehouse rows using exact BigQuery schema column names."""
7     df = df.copy()
8
9     # 1. Timeline Sorting (Using your official warehouse column: transaction_month)
10    df['month_date'] = pd.to_datetime(df['transaction_month'] + "-01", errors='coerce')
11    df = df.sort_values('month_date').reset_index(drop=True)
12
13    # 2. Extract Calendar Numerical Tokens
14    df['transaction_year'] = df['month_date'].dt.year
15    df['transaction_month'] = df['month_date'].dt.month
16
17    # 3. Handle Storey Non-Linear Mappings
18    df["floor_level"] = df["storey_range"].str.split(" ").str[0].astype(float)
19
20    # 4. MOP Lease Shock Waves (Using your official warehouse column: remaining_lease_years)
21    df['remaining_lease_numeric'] = pd.to_numeric(df['remaining_lease_years'],
errors='coerce').fillna(85.0)
22    df['lease_commence_date'] = pd.to_numeric(df['lease_commence_date'],
errors='coerce').fillna(df['transaction_year'] - (99.0 - df['remaining_lease_numeric']))
23
24    # 5. Granular Micro Lags Calculation [Town + Flat Type]
25    # 🌟 FIXED: Added chronological sorting to guarantee safe, sequential shifting!
26    town_type_monthly = (
27        df.groupby(['month_date', 'town', 'flat_type'])['resale_price']
28        .mean()
29        .reset_index()
30        .sort_values('month_date')
31    )
32
33    town_type_monthly = town_type_monthly.rename(columns={'resale_price': 'grid_avg_price'})
34
35    # Generate your leakage-proof shifting momentum anchors
36    town_type_monthly['lag_1'] = town_type_monthly.groupby(['town', 'flat_type'])
['grid_avg_price'].shift(1)
37    town_type_monthly['lag_12'] = town_type_monthly.groupby(['town', 'flat_type'])
['grid_avg_price'].shift(12)
38
39    # Add a 3-month moving average of past months to trace trajectory velocity
40    town_type_monthly['roll_mean_3'] = (
41        town_type_monthly.groupby(['town', 'flat_type'])['lag_1']
42        .transform(lambda x: x.rolling(3, min_periods=1).mean())
43    )
44
45    # Impute initial historical boundary gaps using the global dataset median
46    global_median = df['resale_price'].median()
47    for col in ['lag_1', 'lag_12', 'roll_mean_3']:
48        town_type_monthly[col] = town_type_monthly[col].fillna(global_median)

```

```
49
50     # Left-merge your clean time-series columns back into the master dataset
51     df = pd.merge(df, town_type_monthly.drop(columns=['grid_avg_price']), on=['month_date', 'town',
'flat_type'], how='left')
52
53     # 6. Safe Proximity Conversions & Outlier Handlers
54     proximity_metrics = [
55         "dist_to_closest_mrt_km", "dist_to_closest_lrt_km", "dist_to_closest_shopping_mall_km",
56         "dist_to_closest_primary_school_km", "min_distance_to_regional_hub_km"
57     ]
58     for col in proximity_metrics:
59         df[col] = pd.to_numeric(df[col], errors='coerce').fillna(df[col].median())
60
61
62     return df
63
```